

DLCA Lab 3: Performance

Week 2

August 26, 2025

Introduction

In this week's lab, you shall be learning how to optimize code by exploiting microarchitecture. You shall learn to find the "hotspots" in your code and tune them to be faster.

Necessary tools

We shall be using gcc and g++ in order to compile C and C++ code.

Before starting the lab, ensure that the tool is installed in your system. This can be verified by running the following:

```
$ gcc
gcc: fatal error: no input files
compilation terminated.
$ g++
g++: fatal error: no input files
compilation terminated.
```

Structure of submission/templates

```
your-roll-no/
|- task1/
    |- heat-map.cpp (TODO)
|- task2/
    |- repeated-array.cpp (TODO)
```

```
| - task3/  
    | - weird-matrix.cpp (TODO)
```

The folder structure of the expected submission for this lab is given above. We expect this folder to be compressed into a tarball with the naming convention of your-roll-no.tar.gz. The command given below can be used to do the same

```
$ tar -cvzf your-roll-no.tar.gz your-roll-no/
```

Tasks

This assignment consists of three subtasks. This task is expected to be submitted out-lab. The general structure of this lab is that you have been given various codes that are either inefficient or have scope for improvement. Your job is to figure out and improve those codes. You must compile the codes using:

```
g++ -O0 <code-name>.cpp -o <code-name>
```

Furthermore, you may **not** use any OS or compiler related optimizations (for example, -O3 or -fopenmp), and we will be running your codes set to only one CPU core.

Task 1: Aim to Optimize

You've been given an algorithm where we perform a simulation of heat spread across a 2-D surface. The exact working of this code is not necessarily relevant to the lab. However the general idea is that you iterate over a matrix (which represents the heat values of all points on the 2D surface) and recompute the heat transfer from adjacent points for a fixed number of steps. Further there are randomly picked hotspots which get hotter over time (a step is a unit of time in this case). The final heat values of the grid are dumped to a csv file under the name 'final grid.csv' The code is unoptimised in a few ways, and the task is to figure out these bottle necks and *fix them*.

You should proceed as follows:

Use either perf or AMD μ Prof or Intel Vtune, depending on your platform to analyze the "hotspots" in the code. Analyze various statistics, cache misses, instructions, branch prediction stats, etc., and find out what is causing the code to be less than optimal

Your job is to then eliminate the bottlenecks and write the optimized code in a file

called `heat-map.cpp`. We expect that there are at least two bottlenecks causing a slowdown.

Task 2: What a predict-ament!

You've been given an algorithm where we run a for loop over the elements of an array and based on the value of the element we perform some computation. Furthermore, the values of the array have been randomly initialized. To make the timings more reliable, we perform the computation 150,000 times. To prevent cheating by, say, memoization, we scale our computation in each iteration by the iteration itself. Note: You are explicitly forbidden from computing the result of one iteration and then multiplying it by $\frac{n(n+1)}{2}$, that is against the spirit of the assignment.

You have been given some comments in the code, you are only allowed to write in the allotted spaces. In particular, you have been given two spaces, in the first, you may set up and initialize any variables you wish to use. You should not be performing any computation here. In the second, you have to perform your computations.

You should proceed as follows:

1. Use either `perf` or AMD `μProf` or Intel `Vtune`, depending on your platform to analyze the "hotspots" in the code. Analyse various statistics - cache misses, instructions, branch prediction stats etc and find out what is causing the code to be less than optimal
2. Write your code, in the designated spaces, and make sure your code is faster than the baseline. We print timing numbers to the terminal, lower is better
3. Check and make sure that all your changes are based on microarchitectural phenomena and none based on the general algorithm, any changes to the algorithm go against the spirit of this lab

For a bit of context, although this will vary from laptop to laptop, on the problem setter's laptop, the naive implementation takes around 6600 milliseconds to run and his naive solution takes around 1300 milliseconds. You will be given full marks if your solution takes less than 3000 milliseconds to run. And otherwise, you will be given a linearly decreasing proportion of marks.

Task 3: Matrices Again

You are given an algorithm which uses matrices and performs some computation using the values of those matrices. Much like in the previous task, what compu-

tation is performed, is decided using the value of an array with random elements. Once again, your task is to make this code faster.

Unlike the previous task, you have been given complete freedom to make whatever changes you want.

You should proceed as follows:

1. Use either perf or AMD μ Prof or Intel Vtune, depending on your platform to analyze the "hotspots" in the code. Analyse various statistics - cache misses, instructions, branch prediction stats etc and find out what is causing the code to be less than optimal
2. Write your code, in the designated spaces, and make sure your code is faster than the baseline. We print timing numbers to the terminal, lower is better

For this task you will be given marks based on yours and your peer's performance, only the fastest code will get full marks! For context, the naive code takes around 12700 milliseconds to run on the problem setter's laptop and his naive solution takes around 3900 milliseconds to run.